

可以。我们这次只走一条线：

Policy Gradient → Baseline → Actor-Critic → Advantage → A2C → TRPO → PPO

重点不是背公式，而是搞清楚：

每一步到底发现了什么问题，又是怎么解决这个问题的。

1. 先从 RL 的最终目标开始

我们有一个策略：

$$\pi_{\theta}(a|s)$$

其中：

- s : state, 状态
- a : action, 动作
- θ : 策略网络参数
- $\pi_{\theta}(a|s)$: 在状态 s 下选择动作 a 的概率

我们希望找到一个最优策略：

$$\pi_{\theta^*}$$

使得长期累计奖励最大。

定义 trajectory：

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$$

trajectory 的 return：

$$R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t$$

于是 RL 的目标：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

也就是：

让策略产生的 trajectory 平均回报最大。

2. 为什么不能直接对 $J(\theta)$ 求导？

这是 Policy Gradient 最重要的地方。

我们写：

$$J(\theta) = \sum_{\tau} P_{\theta}(\tau) R(\tau)$$

于是：

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} P_{\theta}(\tau) R(\tau)$$

使用一个非常重要的恒等式：

$$\nabla_{\theta} P_{\theta}(\tau) = P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau)$$

所以：

$$\nabla_{\theta} J(\theta) = \sum_{\tau} P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau) R(\tau)$$

写成期望：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau) \nabla_{\theta} \log P_{\theta}(\tau)]$$

这就是 Policy Gradient 的基础。

3. trajectory probability 怎么展开？

一条 trajectory：

$$\tau = (s_0, a_0, s_1, a_1, \dots)$$

它出现的概率：

$$P_{\theta}(\tau) = \rho(s_0) \prod_t \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

其中：

- $\rho(s_0)$ ：初始状态分布
- $\pi_{\theta}(a_t | s_t)$ ：策略
- $P(s_{t+1} | s_t, a_t)$ ：环境转移概率

取 log：

$$\log P_{\theta}(\tau) = \log \rho(s_0) + \sum_t \log \pi_{\theta}(a_t | s_t) + \sum_t \log P(s_{t+1} | s_t, a_t)$$

注意：

$$\rho$$

和：

$$P(s_{t+1}|s_t, a_t)$$

都与 θ 无关。

因此：

$$\nabla_{\theta} \log P_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$$

于是：

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right]$$

4. 从 trajectory return 变成 Q

这里还可以进一步简化。

在时间 t 的动作 a_t ，它不应该受到过去 reward 的影响。

真正和它有关的是：

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

所以：

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t G_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right]$$

而：

$$Q^{\pi}(s_t, a_t) = \mathbb{E}[G_t|s_t, a_t]$$

所以：

$$\nabla_{\theta} J(\theta) = \mathbb{E}[Q^{\pi}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)]$$

这就是经典的 **Policy Gradient Theorem**。

5. 这句话一定要理解

:

$$\nabla_{\theta} J = \mathbb{E}[Q(s, a) \nabla_{\theta} \log \pi(a|s)]$$

它告诉我们：

如果：

$$Q(s, a) > 0$$

那么：

$$\nabla_{\theta} \log \pi(a|s)$$

会推动这个动作概率增加。

如果：

$$Q(s, a) < 0$$

就会推动它下降。

因此：

Q 就像是在给 Actor 打分

6. 但是这里出现第一个巨大问题：Variance

假设：

$$Q(s, a) = 100$$

这个动作到底好不好？

不知道。

因为可能：

$$V(s) = 99$$

那么这个动作其实只是：

$$100 - 99 = 1$$

比平均水平好一点。

也可能：

$$V(s) = 10$$

那这个动作就非常好了：

$$100 - 10 = 90$$

所以直接使用 Q 存在很大的方差。

7. 第一次重大改进：Baseline

Policy Gradient 有一个非常漂亮的性质：

我们可以减去一个只依赖于状态的 baseline：

$$b(s)$$

而不改变期望梯度。

即：

$$\nabla_{\theta} J = \mathbb{E}[(Q(s, a) - b(s)) \nabla_{\theta} \log \pi(a|s)]$$

只要：

$$b(s)$$

不依赖于动作 a 。

8. 为什么可以随便减 baseline?

这是一个非常值得理解的数学结论。

考虑：

$$\mathbb{E}[b(s) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

对固定 s ：

$$\sum_a \pi_{\theta}(a|s) b(s) \nabla_{\theta} \log \pi_{\theta}(a|s)$$

因为：

$$\nabla \log \pi = \frac{\nabla \pi}{\pi}$$

所以：

$$= b(s) \sum_a \nabla_{\theta} \pi_{\theta}(a|s)$$

而：

$$\sum_a \pi_{\theta}(a|s) = 1$$

所以：

$$\sum_a \nabla_{\theta} \pi_{\theta}(a|s) = \nabla_{\theta} 1 = 0$$

因此：

$$\mathbb{E}[b(s) \nabla \log \pi(a|s)] = 0$$

所以减 baseline 不会改变 Policy Gradient 的期望。

9. 最好的 baseline 是什么？

直觉上：

最好的 baseline 应该告诉我们“这个状态正常情况下能得到多少”。

这正是：

$$V^{\pi}(s)$$

于是：

$$b(s) = V^{\pi}(s)$$

得到：

$$Q^{\pi}(s, a) - V^{\pi}(s)$$

定义：

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

这就是 **Advantage Function**。

于是 Policy Gradient 变成：

$$\nabla_{\theta} J = \mathbb{E}[A^{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

10. Advantage 到底是什么意思？

一定要把它彻底理解。

$$Q(s, a)$$

问：

我在 s 做 a ，结果怎么样？

而：

$$V(s)$$

问：

我在 s ，按照当前策略平均来说怎么样？

所以：

$$A(s, a) = Q(s, a) - V(s)$$

问：

这个动作比“正常水平”好多少？

例如：

$$V(s) = 50$$

三个动作：

$$Q(s, a_1) = 80$$

$$Q(s, a_2) = 55$$

$$Q(s, a_3) = 20$$

那么：

$$A(s, a_1) = 30$$

$$A(s, a_2) = 5$$

$$A(s, a_3) = -30$$

所以：

a1: +30 → 强烈增加概率
a2: +5 → 稍微增加概率
a3: -30 → 降低概率

这就是 Actor 真正需要的信息。

11. 第二次重大改进：Actor-Critic

问题来了：

我们知道 Advantage 很好，但是 Q 和 V 怎么得到？

直接用完整 trajectory 的：

$$G_t$$

当然可以。

但是 variance 仍然比较大。

于是：

Critic

出现了。

让一个神经网络：

$$V_{\phi}(s)$$

学习：

$$V^{\pi}(s)$$

然后：

$$A(s, a)$$

就可以被估计。

这就是：

$$\boxed{Actor - Critic}$$

12. Actor-Critic 的两个网络

Actor

$$\pi_{\theta}(a|s)$$

负责：

做什么？

Critic

$$V_{\phi}(s)$$

负责：

这个状态怎么样？

所以：

$$\boxed{Actor = Policy}$$

$$\boxed{Critic = Value}$$

13. Advantage 怎么估计？

最简单的方法就是 TD。

Bellman equation：

$$V^\pi(s_t) = \mathbb{E}[r_t + \gamma V^\pi(s_{t+1})]$$

所以：

$$r_t + \gamma V(s_{t+1})$$

可以作为当前状态价值的 target。

于是 TD error：

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

这个东西可以作为 Advantage 的估计：

$$A_t \approx \delta_t$$

14. 为什么 TD error 可以近似 Advantage?

从：

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

开始。

如果用一步 TD：

$$Q(s_t, a_t) \approx r_t + \gamma V(s_{t+1})$$

所以：

$$A_t \approx r_t + \gamma V(s_{t+1}) - V(s_t)$$

正好：

$$A_t \approx \delta_t$$

15. A2C 的 Actor 更新

于是 Actor loss：

$$L_{\text{actor}} = -\mathbb{E}_t[\log \pi_\theta(a_t|s_t) A_t]$$

为什么这个公式合理？

假设：

$$A_t > 0$$

那么：

$$-\log \pi(a_t|s_t)A_t$$

希望变小。

因此：

$$\log \pi(a_t|s_t)$$

变大。

所以：

$$\pi(a_t|s_t)$$

增加。

反之：

$$A_t < 0$$

概率下降。

16. Critic 更新

Critic 要预测：

$$V_\phi(s_t)$$

例如使用 TD target：

$$y_t = r_t + \gamma V_\phi(s_{t+1})$$

于是：

$$L_V = \frac{1}{2}(y_t - V_\phi(s_t))^2$$

17. 这就是 A2C 的基本结构

最终：

$$L = L_{\text{actor}} + c_v L_V - c_e H$$

其中：

$$L_{\text{actor}} = -\log \pi(a_t|s_t)A_t$$

$$L_V = \frac{1}{2}(y_t - V(s_t))^2$$

$$H = - \sum_a \pi(a|s) \log \pi(a|s)$$

Entropy 用来鼓励探索。

18. 到这里出现 PPO 前最关键的问题

A2C:

```
旧策略  $\pi_{old}$ 
  ↓
采样数据
  ↓
更新一次
  ↓
新策略  $\pi_{new}$ 
```

问题:

这些数据能不能继续用?

通常不行。

因为数据是从:

π_{old}

采出来的。

但是你更新后, 已经变成:

π_{new}

所以:

$(s, a) \sim \pi_{old}$

而你现在却拿它来优化:

π_{new}

这就是 **on-policy** 的问题。

19. 为什么 PPO 非常重要?

如果每采一批数据只能更新一次:

采样 1000 条
↓
更新一次
↓
扔掉
↓
重新采样

样本利用率非常低。

所以 PPO 的核心问题：

能不能让一批旧数据使用多次？

但如果直接使用，会有一个问题：

新策略可能跑得离旧策略太远。

所以 PPO 的核心思想就是：

允许多次更新 + 限制新旧策略不能变化太大

20. 这个思想其实来自 TRPO

PPO 前面有一个非常重要的算法：

TRPO

Trust Region Policy Optimization。

它的核心思想：

每次更新 Policy 时，只允许它在旧 Policy 附近移动。

数学上可以写：

$$\max_{\theta} \mathbb{E} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\text{old}}(a|s)} A^{\pi_{\text{old}}}(s, a) \right]$$

同时约束：

$$D_{KL}(\pi_{\text{old}} \parallel \pi_{\theta}) \leq \delta$$

这就是所谓：

Trust Region

21. 为什么要限制 KL？

考虑：

$$A(s, a) > 0$$

说明动作 a 好。

那么我们当然希望：

$$\pi_{\theta}(a|s)$$

增加。

但是如果无限增加：

$$0.1 \rightarrow 0.9$$

可能就太激进。

因为 Critic 对 Advantage 的估计本身可能是错的。

例如：

$$A(s, a) = 10$$

实际上可能只是：

$$A(s, a) = 1$$

如果你一次更新直接把：

$$\pi(a|s) : 0.1 \rightarrow 0.9$$

就可能把一个错误的估计放大。

所以：

Policy update 不能太激进

22. TRPO 的问题

TRPO 理论非常漂亮：

$$\max L(\theta)$$

subject to:

$$KL(\pi_{\text{old}}, \pi_{\theta}) < \delta$$

但实现比较复杂。

它需要：

- KL constraint
- 二阶信息

- Fisher information matrix
- Conjugate Gradient
- line search

所以工程上比较麻烦。

于是 PPO 出现：

能不能用一个简单的一阶优化方法，实现类似 Trust Region 的效果？

23. PPO 的核心：Probability Ratio

定义：

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

这个 ratio 非常重要。

例如：

旧策略：

$$\pi_{\text{old}}(a|s) = 0.2$$

新策略：

$$\pi_\theta(a|s) = 0.3$$

那么：

$$r_t = 1.5$$

说明：

新策略让这个动作的概率增加了 50%。

如果：

$$r_t = 0.5$$

说明：

新策略把概率降低了一半。

24. 如果 Advantage 是正的

假设：

$$A_t = 10$$

这个动作很好。

PPO 希望：

$$r_t > 1$$

也就是：

$$\pi_{\theta}(a_t|s_t) > \pi_{\text{old}}(a_t|s_t)$$

所以：

$$r_t A_t$$

希望尽量大。

25. 如果 Advantage 是负的

假设：

$$A_t = -10$$

这个动作不好。

PPO 希望：

$$r_t < 1$$

也就是：

$$\pi_{\theta}(a_t|s_t) < \pi_{\text{old}}(a_t|s_t)$$

这样：

$$r_t(-10)$$

会变得更大。

所以：

$$A > 0 \Rightarrow \text{增加概率}$$

$$A < 0 \Rightarrow \text{减少概率}$$

26. 但是还不能让它无限变化

所以 PPO 引入：

$$\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$$

比如：

$$\epsilon = 0.2$$

那么：

$$r_t$$

只能有效地处在：

$$[0.8, 1.2]$$

附近。

PPO 的核心目标：

$$L^{CLIP} = \mathbb{E}_t[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

这是整个 PPO 最重要的公式。

27. 为什么是 min?

这是 PPO 最容易被搞混的地方。

我们分两种情况。

情况一： $A_t > 0$

动作是好的。

我们希望：

$$r_t$$

增加。

假设：

$$A_t = 10$$

并且：

$$r_t = 1.5$$

那么：

$$r_t A_t = 15$$

但是 clip：

$$\text{clip}(1.5, 0.8, 1.2) = 1.2$$

所以：

$$1.2A_t = 12$$

于是：

$$\min(15, 12) = 12$$

也就是说：

最多奖励你把这个好动作的概率增加到 1.2 倍。

再增加没有收益。

28. 情况二： $A_t < 0$

假设：

$$A_t = -10$$

动作是坏的。

现在如果：

$$r_t = 0.5$$

说明我们成功降低了这个动作概率。

正常：

$$r_t A_t = 0.5(-10) = -5$$

clip：

$$\text{clip}(0.5, 0.8, 1.2) = 0.8$$

所以：

$$0.8(-10) = -8$$

那么：

$$\min(-5, -8) = -8$$

这里注意：

$$-8 < -5$$

因此选择：

$$-8$$

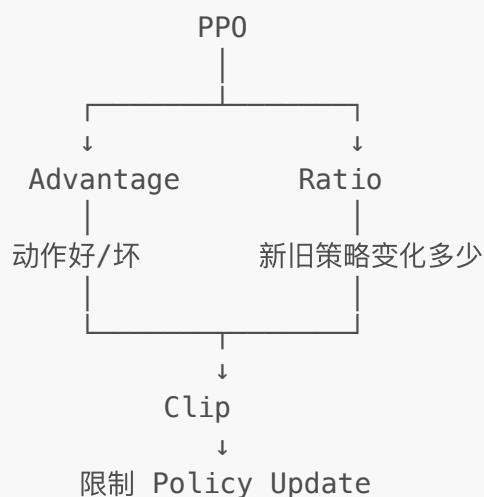
从而防止继续通过极端降低概率获得更大的优化收益。

29. PPO clipping 到底在做什么？

一句话：

如果新策略和旧策略差得太远，就不再奖励这种继续变远的行为。

所以：



30. 所以从 A2C 到 PPO，真正发生了什么？

现在把整个演化路线串起来：

第一阶段：Policy Gradient

直接：

$$\nabla J = E[Q \nabla \log \pi]$$

问题：

Variance 很大

第二阶段：Baseline

减去：

$$b(s)$$

得到：

$$\nabla J = E[(Q - b(s)) \nabla \log \pi]$$

降低 variance。

第三阶段：Advantage

选择：

$$b(s) = V(s)$$

得到：

$$A(s, a) = Q(s, a) - V(s)$$

于是：

$$\nabla J = E[A \nabla \log \pi]$$

第四阶段：Actor-Critic

用一个网络学习：

$$V_{\phi}(s)$$

然后估计：

$$A_t$$

于是：

$$Actor : \pi_{\theta}(a|s)$$

$$Critic : V_{\phi}(s)$$

第五阶段：A2C

使用：

$$A_t \approx r_t + \gamma V(s_{t+1}) - V(s_t)$$

进行同步并行采样和更新。

但是：

$$\text{On-policy}$$

导致：

$$\text{样本利用率低}$$

第六阶段：TRPO

提出：

可以多次利用数据，但新策略不能离旧策略太远。

使用：

$$KL(\pi_{\text{old}}, \pi_{\theta}) < \delta$$

限制更新。

但是：

实现复杂

第七阶段：PPO

用：

$$r_t = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

衡量新旧策略变化。

然后：

$$L^{CLIP} = E[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

用 clipping 简单地限制 Policy update。

31. PPO 的完整 Loss

实际 PPO 通常不只有 Actor。

完整目标大致是：

$$L = L_{\text{policy}} + c_v L_{\text{value}} - c_e L_{\text{entropy}}$$

其中：

Policy

$$L_{\text{policy}} = -E[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

注意实现中通常是最小化 **loss**，所以前面有负号。

Value

例如：

$$L_{\text{value}} = \frac{1}{2}(V_{\phi}(s_t) - V_t^{\text{target}})^2$$

Entropy

$$L_{\text{entropy}} = H(\pi_{\theta})$$

最终：

$$L_{\text{total}} = L_{\text{policy}} + c_v L_{\text{value}} - c_e H$$

32. 还有一个非常重要的东西：GAE

如果你真正想理解现代 PPO，**GAE 必须搞懂**。

因为 PPO 里面的：

$$A_t$$

通常不是简单地：

$$A_t = \delta_t$$

而是使用：

$$\boxed{\text{GAE}}$$

Generalized Advantage Estimation。

先定义：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

然后：

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + \dots$$

也就是：

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

33. 为什么 GAE 好？

因为：

只用一步 TD

$$A_t \approx \delta_t$$

优点：

低 variance

缺点：

高 bias

而如果直接使用完整 return：

$$A_t \approx G_t - V(s_t)$$

优点：

低 bias

缺点：

高 variance

GAE 通过：

$$\lambda$$

在二者之间进行 trade-off：

$$\lambda \rightarrow 0 \Rightarrow \text{更像 TD}$$

$$\lambda \rightarrow 1 \Rightarrow \text{更像 Monte Carlo}$$

所以：

$$GAE = \text{Bias-Variance Tradeoff}$$

34. 现在把 PPO 的整个训练流程串起来

假设我们有当前策略：

$$\pi_{\theta_{\text{old}}}$$

Step 1: 采样

用旧策略与环境交互：

$$s_t \rightarrow a_t \rightarrow r_t \rightarrow s_{t+1}$$

得到：

$$(s_t, a_t, r_t, s_{t+1})$$

Step 2: Critic 估计

计算：

$$V(s_t)$$

和：

$$V(s_{t+1})$$

Step 3: 计算 TD error

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Step 4: GAE

计算：

$$\hat{A}_t = \sum_l (\gamma \lambda)^l \delta_{t+l}$$

Step 5: 计算 return / value target

通常：

$$\hat{R}_t = \hat{A}_t + V(s_t)$$

作为 Critic 的训练 target。

Step 6: 保存旧策略概率

记录：

$$\log \pi_{\text{old}}(a_t | s_t)$$

Step 7: 更新 Actor

当前策略重新计算：

$$\log \pi_{\theta}(a_t | s_t)$$

然后：

$$r_t = \exp(\log \pi_{\theta} - \log \pi_{\text{old}})$$

Step 8: Clipped Objective

计算：

$$\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)$$

Step 9: 同时更新 Critic

:

$$L_V = (V(s_t) - \hat{R}_t)^2$$

Step 10: 重复多个 epoch

这就是 PPO 最重要的优势：

```
采样一次
  ↓
得到 batch
  ↓
Epoch 1
  ↓
Epoch 2
  ↓
Epoch 3
  ↓
Epoch 4
  ↓
...
  ↓
丢掉这批数据
  ↓
重新采样
```

也就是说：

PPO 仍然是on-policy

但它通过 clipping：

让一批数据可以安全地使用多个epoch

35. 这里有一个非常容易产生的误解

很多人会说：

“PPO 是 off-policy，因为它可以重复使用数据。”

这是错的。

PPO 本质上还是：

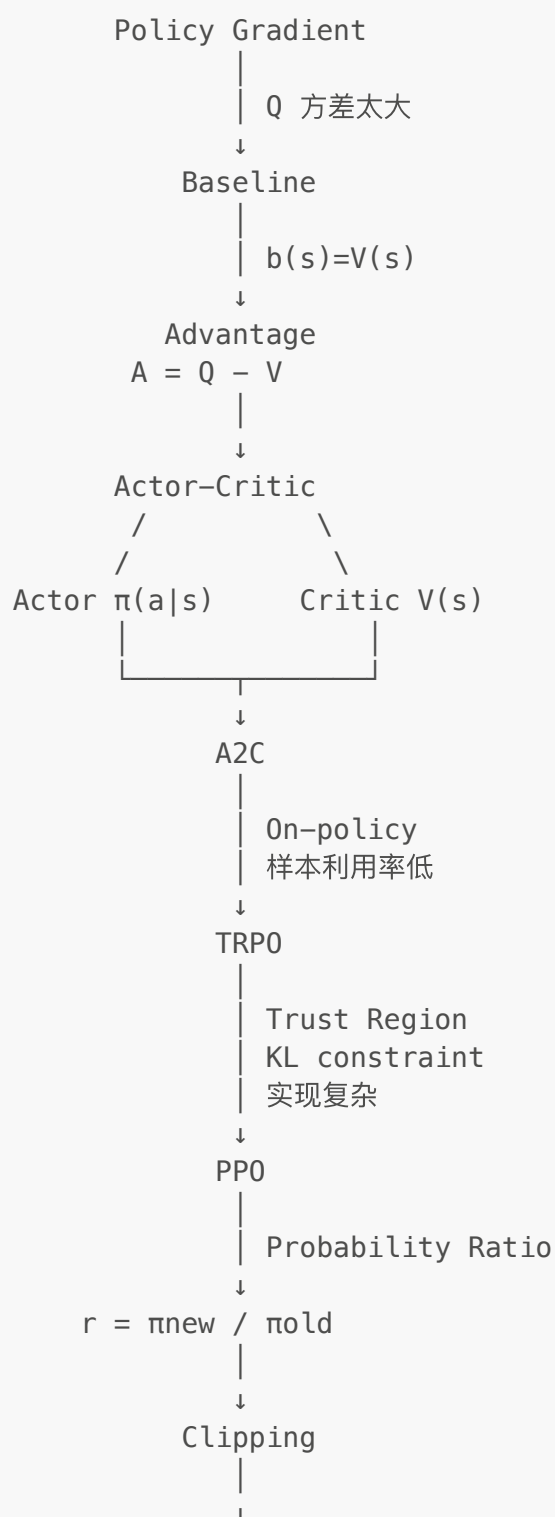
On-policy

它只是：

比传统 Policy Gradient / A2C 更充分地利用当前 batch 的数据。

它并不像 SAC、TD3 那样维护 replay buffer，然后长期重复使用历史经验。

36. 最后用一张图把整个故事串起来



限制 Policy 更新幅度

↓
PPO

37. 你可以把 PPO 记成三个核心公式

如果只让我让你记三个公式，我会选择：

第一：Policy Gradient

$$\nabla_{\theta} J = E[A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

它回答：

为什么 Actor 要根据 Advantage 更新？

第二：Probability Ratio

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

它回答：

新策略相对于旧策略改变了多少？

第三：PPO Clip

$$L^{CLIP} = E[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

它回答：

怎么让策略变好，但又不能一步变得太激进？

38. 最核心的思想其实只有一句话

从 Policy Gradient 一路到 PPO，整个历史可以浓缩成：

“如何让好动作概率增加、坏动作概率降低？”

然后一步步解决问题：

$$Q \rightarrow Q - V = A$$

解决：

怎么降低 Policy Gradient 的 variance?

然后:

$$A \rightarrow V_{\phi}(s)$$

解决:

怎么估计 Advantage?

然后:

$$\pi_{\text{old}} \rightarrow \pi_{\theta}$$

解决:

怎么提高样本利用率?

最后:

$$\frac{\pi_{\theta}}{\pi_{\text{old}}} \rightarrow \text{clip}$$

解决:

怎么防止策略更新过头?

所以 PPO 的真正逻辑不是“突然出现一个 clip 公式”，而是:

Policy Gradient $\rightarrow$ 降低方差 $\rightarrow$ 估计 Advantage $\rightarrow$ 提高样本利用率 $\rightarrow$ 限制策略更新

这条线理解清楚之后，PPO 的 Actor loss、Critic loss、GAE、ratio、clip、old policy、multiple epochs 就全部能放到各自的位置上，而不是一堆孤立公式。